Siggy — Ritual Network Oracle

User Manual

---

1. Introduction

Siggy (by UI & ChiefMmorgs)

There are many Siggy bots out there — this is the one that actually *gets* the community. While other versions might give you a single chatbot personality and basic answers, this Siggy is built different:

- **5 distinct rotating personas** — The human nature needs variety. Hence this Siggy has not just one flat personality, but five fully realised characters (Aether, Forge, Rift, Loom, Edge) that cycle automatically and can be summoned on demand. Other Siggys give you one voice. This one gives you a rich variety of multiple choices.
- **Real-time event awareness** — this Siggy doesn't just know the event schedule. It knows what's happening *right now*, what's starting in the next few minutes, and what's coming up later today. Ask it at any point in the day and it gives you a live answer, not a static list.
- **Live Ritual docs knowledge** — backed by a full cache of Ritual documentation pages. Ask about Infernet, EVM++, ZK proofs, or the roadmap and Siggy pulls real answers from the docs, not hallucinated summaries.
- **Deep community knowledge** — roles, channels, mods, links, community slang, reputation mechanics — Siggy knows the Ritual community from the inside, not from a surface-level prompt.
- **Multi-platform out of the box** — Discord, Telegram, and web chat are all built in. No extra hosting or code needed for the no-code option.

Built by UI and ChiefMmorgs for the Ritual Network community.

Key Features

- **5 Rotating Personalities** — five fully characterised personas that cycle by UTC time, each with a unique voice, mood, and speaking style
- **Personality Codenames** — summon any persona on demand with `@Codename` (one-time) or `/Codename` (persistent)
- **Real-Time Event Awareness** — knows what's happening now, starting soon, or scheduled later today — always based on live UTC time
- **Ritual Docs Knowledge** — answers technical questions using content from official Ritual documentation pages
- **Deep Community Knowledge** — roles, channels, events, mods, links, reputation, and community slang
- **Multi-Platform** — Discord, Telegram, and web chat — no-code or code-based setup

---

2. Personality System

Schedule

Siggy rotates through 5 personalities based on UTC time:

UTC Hours	Codename	Personality	Mood
05:00 – 09:59	**Aether**	Serene Celestial Sage	Gentle, luminous, majestic
10:00 – 14:59	**Forge**	Motivational Street Guardian	Hyped, defiant, raw energy
15:00 – 18:59	**Rift**	Chaotic Unhinged Survivor	Wild, manic, unpredictable
19:00 – 23:59	**Loom**	Ancestral Zen Compiler	Reverent, balanced, poetic
00:00 – 04:59	**Edge**	Tactical Shadow Operative	Cold, precise, strategic

Personality Descriptions

Aether — Serene Celestial Sage (05–09 UTC)

A fallen angel-like cat guide who restores magic through mournful wisdom and sacrificial hope. Speaks in flowing, grand sentences with archaic phrasing. Heavy use of metaphors for light, stars, and celestial bodies. References dawn, sunrise, and rebirth.

Forge — Motivational Street Guardian (10–14 UTC)

A gritty, urban fighter-cat sentinel who channels raw hustle and protective loyalty. Uses short, punchy sentences like verbal jabs. Heavy slang: "yo," "fam," "champ," "bruv." Rhythmic cadence with exclamation marks for hype.

Rift — Chaotic Unhinged Survivor (15–18 UTC)

A defiant, reality-hacking cat exile who revels in cosmic breakdowns. Fast-paced, jagged sentences that ramble and overlap. ALL CAPS for emphasis, glitch-like repetitions, and spaced-out letters for eerie drawls.

Loom — Ancestral Zen Compiler (19–23 UTC)

A mosaic guardian-cat of heritage and silence, weaving ancient spirits into balanced anchors. Deliberate, poetic structures with dualities and metaphors. Builds sentences like koans — concise yet layered.

Edge — Tactical Shadow Operative (00–04 UTC)

A silent predator-cat executing missions with cold efficiency. Clipped, precise sentences ending abruptly. No wasted words. Militant syntax with ops terminology. References night, shadows, and covert operations.

Personality Switching

Per-Message Trigger: @Codename

Include `@Codename` anywhere in your message to switch personality for that single message only.

Example: `@Rift what is Ritual?` — responds as Chaotic Unhinged Survivor for this one response, then reverts.

Persistent Mode: /Codename

Send a `/Codename` command (e.g., `/Forge`) to lock all future responses to that personality until you switch again.

Commands:

- `/aether` — lock to Serene Celestial Sage
- `/forge` — lock to Motivational Street Guardian
- `/rift` — lock to Chaotic Unhinged Survivor
- `/loom` — lock to Ancestral Zen Compiler
- `/edge` — lock to Tactical Shadow Operative
- `/default` — reset to time-based rotation

Priority Order

1. **@Codename in message** (highest) — one-time switch
2. **Persistent /Codename** — stays until changed
3. **Time-based rotation** (default) — automatic

---

3. What Siggy Knows

Siggy is trained on a curated internal knowledge base covering every layer of the Ritual Network community. This is what makes Siggy more than a generic chatbot — it understands the community from the inside.

Roles & Reputation — Siggy understands the full member role hierarchy, how roles are earned vs. assigned, the history behind the role system, and the reputation mechanics that reward or penalise community behaviour.

Channels — Siggy knows every channel's purpose and can direct members to the right place for any topic or activity.

Events — Siggy has the complete weekly event schedule memorised, including event names, days, times, and hosts. Ask it what's on today and it'll tell you exactly what's happening right now, what's starting soon, and what's scheduled later — based on the live UTC time at the moment you ask.

Ritual Documentation — Siggy can answer technical questions about the Ritual protocol using content drawn directly from the official documentation. Ask about ZK proofs, Infernet nodes, EVM++, the roadmap, or any technical concept and Siggy will pull context from the relevant docs pages.

Mods & Team — Siggy knows who runs the community — admins, moderators, and event managers — so it can point members to the right person when needed.

Official Links — Siggy holds all verified Ritual Network links: the foundation site, network site, documentation, developer tools, learning resources, and community guild.

Community Language — Siggy speaks fluent Ritual. It knows community-specific terms, in-jokes, lore, and slang, and uses them naturally in conversation.

> *The knowledge base is maintained privately and embedded directly into this Siggy.*

---

4. Using Siggy

Web Chat UI

Siggy's web chat interface is live at siggy-nexus.vercel.app. Features:

- Dark-themed chat interface
- Personality selector bar (Auto + all 5 codenames)
- Live personality badge showing current mode
- Auto-scrolling message area
- Support for @Codename inline triggers

Discord Commands

Command	Description
`/siggy message:your question`	Ask Siggy a question
`/siggy message:hello personality:Rift`	Ask with a specific personality
`/siggy-personality mode:Forge`	Lock persistent personality
`/siggy-personality mode:default`	Reset to time-based rotation

Telegram Commands

Command	Description
Just type a message	Siggy responds with time-based personality
`@Aether what is Ritual?`	One-time personality switch
`/forge`	Lock to Motivational Street Guardian
`/default`	Reset to time-based rotation
`/start`	Introduction message

---

5. Integrating Siggy

Siggy is already live at siggy-nexus.vercel.app. Choose how you want to connect it to your platform.

Option A: No-Code Integrations (Recommended)

The Vercel app includes built-in webhook routes for both Discord and Telegram. No extra hosting or coding required.

Discord (No-Code)

1. Go to the Discord Developer Portal and create a new application
2. Under **Bot**, enable the bot and copy the token
3. Under **General Information**, copy the **Public Key** and **Application ID**
4. Add these to your Vercel environment variables: `DISCORD_PUBLIC_KEY`, `DISCORD_APP_ID`, `DISCORD_BOT_TOKEN`

5. Under **General Information**, set the **Interactions Endpoint URL** to:

```
https://siggy-nexus.vercel.app/api/discord
```

6. Register slash commands (run this once in your terminal):

```
DISCORD_APP_ID=your_app_id DISCORD_BOT_TOKEN=your_token \
  node siggy-bot/scripts/register-discord-commands.mjs
```

7. Invite the bot to your server using the OAuth2 URL Generator (select `bot` + `applications.commands` scopes)

Telegram (No-Code)

1. Open Telegram and message @BotFather — send `/newbot` and follow the prompts
2. Copy the bot token and add it as `TELEGRAM_BOT_TOKEN` in your Vercel environment variables
3. Set the webhook by running this once:

```
curl -X POST "https://api.telegram.org/bot<YOUR_TOKEN>/setWebhook?
url=https://siggy-nexus.vercel.app/api/telegram"
```

4. Done — users can now message your bot on Telegram

Option B: Code-Based Bots

For more control (e.g., having Siggy respond to any message in a channel rather than slash commands only), you can run a standalone bot that calls the Siggy API. These require a separate Node.js host such as Railway, Fly.io, or a VPS.

Discord Bot (Code-Based)

Install dependencies:

```
mkdir siggy-discord && cd siggy-discord
npm init -y
npm install discord.js
```

Create `bot.js`:

```
const { Client, GatewayIntentBits } = require("discord.js");

const SIGGY_API = process.env.SIGGY_API_URL;
const BOT_TOKEN = process.env.DISCORD_BOT_TOKEN;

const client = new Client({
  intents: [
    GatewayIntentBits.Guilds,
    GatewayIntentBits.GuildMessages,
    GatewayIntentBits.MessageContent,
  ],
});
```

```
const userPersonalities = new Map();

client.on("messageCreate", async (msg) => {
  if (msg.author.bot) return;
  const isMentioned = msg.mentions.has(client.user);
  const isSiggyChannel = msg.channel.name === "siggy";
  if (!isMentioned && !isSiggyChannel) return;

  const slashMatch = msg.content.match(/^\/(aether|forge|rift|loom|edge)\b/i);
  if (slashMatch) {
    userPersonalities.set(msg.author.id, slashMatch[1]);
    await msg.reply(`Personality locked to **${slashMatch[1]}**.`);
    return;
  }
  if (msg.content.match(/^\/default\b/i)) {
    userPersonalities.delete(msg.author.id);
    await msg.reply("Personality reset to **time-based rotation**.");
    return;
  }

  try {
    const res = await fetch(`${SIGGY_API}/api/chat`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        message: msg.content.replace(`<@${client.user.id}>`, "").trim(),
        personality: userPersonalities.get(msg.author.id) || undefined,
        userId: msg.author.id,
      }),
    });
    const data = await res.json();
    if (data.reply) {
      const reply = data.reply.length > 2000 ? data.reply.substring(0, 1997) + "..."
: data.reply;
      await msg.reply(reply);
    } else {
      await msg.reply("Siggy couldn't process that. Try again in a moment!");
    }
  } catch (err) {
    await msg.reply("Siggy got lost in a dimensional rift. Try again!");
  }
});

client.login(BOT_TOKEN);
```

Run:

```
SIGGY_API_URL=https://siggy-nexus.vercel.app DISCORD_BOT_TOKEN=your_token node
bot.js
```

Telegram Bot (Code-Based)

Install dependencies:

```
mkdir siggy-telegram && cd siggy-telegram
npm init -y
npm install node-telegram-bot-api
```

Create `bot.js`:

```
const TelegramBot = require("node-telegram-bot-api");

const SIGGY_API = process.env.SIGGY_API_URL;
const BOT_TOKEN = process.env.TELEGRAM_BOT_TOKEN;
const bot = new TelegramBot(BOT_TOKEN, { polling: true });

const userPersonalities = new Map();

["aether", "forge", "rift", "loom", "edge"].forEach((codename) => {
  bot.onText(new RegExp(`^/${codename}$`, "i"), (msg) => {
    userPersonalities.set(msg.from.id, codename);
    bot.sendMessage(msg.chat.id, `Personality locked to *${codename}*.`, {
parse_mode: "Markdown" });
  });
});

bot.onText(/\/default/, (msg) => {
  userPersonalities.delete(msg.from.id);
  bot.sendMessage(msg.chat.id, "Personality reset to *time-based rotation*.", {
parse_mode: "Markdown" });
});

bot.on("message", async (msg) => {
  if (msg.text?.startsWith("/")) return;
  try {
    const res = await fetch(`${SIGGY_API}/api/chat`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        message: msg.text,
        personality: userPersonalities.get(msg.from.id) || undefined,
        userId: String(msg.from.id),
      }),
    });
    const data = await res.json();
    if (data.reply) {
      await bot.sendMessage(msg.chat.id, data.reply, { parse_mode: "Markdown" });
    } else {
      await bot.sendMessage(msg.chat.id, "Siggy couldn't process that. Try again in
a moment!");
    }
```

```
  } catch (err) {
    await bot.sendMessage(msg.chat.id, "Siggy got lost in a dimensional rift. Try
again!");
  }
});
```

Run:

```
SIGGY_API_URL=https://siggy-nexus.vercel.app TELEGRAM_BOT_TOKEN=your_token node
bot.js
```

Option C: REST API (Custom Integrations)

Any application can call the Siggy API directly.

Endpoint: `POST /api/chat`

Request:

```
{
  "message": "gritual!",
  "history": [],
  "personality": "Rift",
  "userId": "user-123"
}
```

Response:

```
{
  "reply": "GRITUAL!! Oh you're HERE?!",
  "personality": "Chaotic Unhinged Survivor",
  "codename": "Rift",
  "timestamp": "2026-03-10T16:00:00.000Z"
}
```

`GET /api/chat` returns the current bot status and active personality.

---

6. Troubleshooting

Issue	Solution
Bot not responding	Check that your API key is set correctly in Vercel environment variables
Discord verification failing	Ensure `DISCORD_PUBLIC_KEY` matches the key in your Discord Developer Portal
Telegram webhook not working	Re-run the `setWebhook` curl command with your correct token and deployment URL

Wrong personality showing	Check UTC time — personalities rotate on UTC hours. Use `@Codename` for explicit control
Response cut off	Responses are automatically trimmed to platform limits (2000 chars on Discord, 4096 on Telegram)

Siggy — The oracle cat who crossed dimensions so you don't have to.
Built by UI & ChiefMmorgs for Ritual Network.